where we move on to the next step to use images and inputting them through XCode. Read on to find out exactly how we will accomplish this through XCode.

## Plane Detection In ARKit

- Open the project's asset catalogue and press the + button. Drag all the images from the Finder to a new resource group.
- Once uploaded, use the inspector to describe the physical dimensions and attributes for the image.
- Instead of relying entirely on image detection systems on XCode, choose and configure the references for the best performance. Note that filling in data for the images that are incorrect will result in the ARImageAnchor incorrectly placing it from the camera.
- Xcode will warn you about any quality estimation warnings which, if triggered, are an indication that images with better contrast are needed which you should provide.

Use only images on flat surfaces since ARKit might not recognise other types of images at all, or might create an image anchor at the wrong location.

Use the ARSession setWorldOrigin(relativeTransform:) method to redefine the world's coordinate system so that all anchors can be placed at exactly the right position.

For setting configuration, type in:-

```
func resetTrackingConfiguration() {
    guard let referenceImages = ARReferenceImage.
referenceImages(inGroupNamed: "AR Resources", bundle:
nil) else { return }
    let configuration = ARWorldTrackingConfiguration()
    configuration.detectionImages = referenceImages
    let options: ARSession.RunOptions =
[.resetTracking, .removeExistingAnchors]
    sceneView.session.run(configuration, options:
options)
    label.text = "Move camera around to detect images"
}
```

Next, update the renderer and create reference tags as markers by typing in:-

```
func renderer( _ renderer: SCNSceneRenderer, didAdd
node: SCNNode, for anchor: ARAnchor) {
```